

Sistema de gestión para local de ropa: Plan de arquitectura (Etapa de diseño)

Proyecto: `isis-gestion` · Stack: React + Vite + TypeScript + Supabase (PostgreSQL, Auth, RLS) · Proyecto independiente, sin código ni lógica reutilizada de otros sistemas.

0. Principios de diseño

- La lógica crítica vive en PostgreSQL, no en React.** Vender, mover stock, abrir o cerrar caja y recibir compras son funciones SQL (RPC) transaccionales y atómicas. El frontend nunca hace `UPDATE stock` ni `INSERT` directo en tablas transaccionales.
- Libros inmutables (ledgers).** `stock_movements`, `cash_movements` y `supplier_ledger` son append-only. No se hace UPDATE ni DELETE. Un error se corrige con un movimiento inverso.
- Los saldos son derivados.** El stock actual es una caché mantenida por trigger a partir de los movimientos y se puede auditar contra ellos. Lo mismo vale para el efectivo esperado y el saldo del proveedor.
- Documentos con foto (snapshot).** Una venta guarda nombre, SKU, EAN, color, talle, precio y costo tal como eran ese día.
- Seguridad en profundidad.** RLS en todas las tablas, permisos verificados dentro de cada función, y el costo aislado de los vendedores.
- Una sola moneda (ARS) y un solo local por ahora,** sin cerrar la puerta a crecer.

1. Arquitectura general

React + Vite + TS	<--HTTPS-->	Supabase
- React Router		Auth (email + password, JWT)
- TanStack Query		PostgREST (lecturas con RLS)
- react-hook-form + zod		PostgreSQL: tablas + RLS, funciones RPC
- Tailwind		(SECURITY DEFINER), triggers, vistas
- SheetJS/ExcelJS (xlsx)		Edge Function (solo alta de usuarios)

Capa	Responsabilidad
React	UI, validación de formularios, lectura/parseo de Excel, generación de archivos Excel
PostgREST	Lecturas (listados, búsquedas, reportes) filtradas por RLS
RPC (PL/pgSQL)	Todas las escrituras con reglas de negocio, en una transacción
Triggers	Mantener <code>stock_levels</code> , bloquear UPDATE/DELETE en ledgers, auditoría
Edge Function	Crear usuarios con la service key, que nunca llega al navegador

Stack concreto: TypeScript estricto con tipos generados (`supabase gen types`); migraciones con Supabase CLI, `seed.sql` y tests SQL con pgTAP; Vitest para lógica del frontend; Playwright más adelante para flujos críticos; Tailwind con componentes propios; TanStack Query para estado del servidor y Context para el carrito del POS.

No se instala al principio: librerías de gráficos, impresión de tickets, lector de código de barras ni i18n.

2. Estructura de carpetas

```
isis-gestion/
|- docs/                arquitectura.md, modelo-datos.md, decisiones/ (ADRs)
|- supabase/
|  |- config.toml
|  |- migrations/      0001_extensions.sql, 0002_auth_roles.sql, ...
|  |- seed.sql         métodos de pago, talles, colores, settings base
|  |- tests/           pgTAP: stock, caja, ventas, compras, RLS
|  '- functions/create-user/ Edge Function (service role)
|- src/
|  |- app/             router, providers, layout, guards de rol
|  |- components/ui/   botones, tablas, modales, input de dinero
|  |- features/
|  |  auth, users, catalog, stock, inventory, pos, cash,
|  |  expenses, suppliers, purchases, reports, excel, settings
|  |  (cada una: api/ hooks/ components/ pages/ schemas/)
|  |- lib/             supabase.ts, money.ts, ean.ts, excel/
|  '- types/database.ts generado, no se edita a mano
|- .github/workflows/ci.yml
'- package.json, vite.config.ts, tsconfig.json, .env.example
```

3. Modelo de base de datos

Convenciones: claves `uuid` (los documentos tienen además un número secuencial legible); dinero `numeric(14,2)`; stock `integer`; fechas `timestamptz`; todas las tablas con `created_at` y `created_by` (y `updated_at/by` si son editables); borrado lógico en maestros.

3.1 Seguridad y configuración

Tabla	Campos clave
profiles	id (= auth.users.id), full_name, role (enum app_role), active, created_at
app_settings	key PK, value jsonb (ej. allow_negative_stock=false, store_name, sale_number_prefix)
audit_log	id, table_name, record_id, action, old_data, new_data, user_id, at

3.2 Catálogo

Tabla	Campos clave
categories	id, name, parent_id, active
brands	id, name, active
colors	id, name (único), hex
sizes	id, name (S, M, 38...), size_group, sort_order
products (modelo)	id, code (único), name, description, category_id, brand_id, supplier_id, active
product_variants	id, product_id, color_id, size_id, sku (único), ean (texto, único si no es nulo), price, min_stock, active. UNIQUE(product_id, color_id, size_id)
variant_costs	variant_id PK, last_cost, avg_cost, updated_at

Tabla	Campos clave
price_history	id, variant_id, old_price, new_price, old_cost, new_cost, changed_by, changed_at

El costo va en una tabla aparte porque RLS filtra filas, no columnas, y en Supabase todos los usuarios comparten el rol `authenticated`. En `variant_costs` una política RLS lo restringe a dueño, encargado y depósito.

3.3 Stock e inventario

Tabla	Campos clave
stock_levels	variant_id PK, quantity, updated_at. Solo la modifica el trigger de stock_movements
stock_movements (ledger)	id, variant_id, movement_type, quantity (con signo), stock_before, stock_after, unit_cost, reason, reference_type, reference_id, notes, created_by, created_at
inventory_counts	id, number, status (draft, counting, review, applied, cancelled), scope, started_at, applied_at, created_by, applied_by, notes
inventory_count_items	id, count_id, variant_id, expected_qty, counted_qty, counted_at, counted_by, difference, notes. UNIQUE(count_id, variant_id)

movement_type: purchase_in, sale_out, sale_return_in, sale_void_in, purchase_return_out, adjustment_in, adjustment_out, inventory_adjustment, import_adjustment, initial_load, damage_out, theft_out, internal_use_out.

3.4 Ventas

Tabla	Campos clave
customers (opcional)	id, name, phone, email, document, notes
sales	id, number, cash_session_id, customer_id, status (completed, voided), subtotal, discount_amount, total, notes, client_request_id (único), created_by, created_at, voided_by, voided_at, void_reason
sale_items	id, sale_id, variant_id, quantity, unit_price, discount_amount, line_total, snapshot: product_name, sku, ean, color, size
sale_item_costs	sale_item_id PK, unit_cost (restringido por RLS)
sale_payments	id, sale_id, payment_method_id, amount, amount_tendered, change_given, installments, reference
sale_returns / sale_return_items	sale_id, ítems devueltos, refund_method_id, refund_amount, reason, restock

3.5 Caja y gastos

Tabla	Campos clave
payment_methods	id, code (cash, transfer, debit, credit, mercadopago, other), name, is_cash, active, sort_order, is_system
cash_registers	id, name, active

Tabla	Campos clave
cash_sessions	id, register_id, status (open, closed), opened_by, opened_at, opening_amount, closed_by, closed_at, expected_cash, counted_cash, cash_difference, closing_notes. Índice único parcial: una sola sesión abierta por caja
cash_movements (ledger)	id, cash_session_id, payment_method_id, kind, direction (in/out), amount (> 0), affects_physical_cash, description, reference_type, reference_id, created_by, created_at
cash_session_totals	session_id, payment_method_id, expected, reported
cash_count_details (opcional)	session_id, denomination, quantity
expense_categories	id, name
expenses	id, date, category_id, description, amount, payment_method_id, paid_from_register, cash_session_id, supplier_id, receipt_ref, created_by, voided_at

cash_movements.kind: opening_float, sale, sale_void, sale_refund, income, expense, withdrawal, supplier_payment, adjustment.

3.6 Proveedores y compras

Tabla	Campos clave
suppliers	id, name, tax_id, contact, phone, email, address, payment_terms_days, notes, active
purchases	id, number, supplier_id, invoice_number, purchase_date, due_date, status (draft, received, cancelled), subtotal, discount_amount, tax_amount, total, paid_amount, payment_status (pending, partial, paid), received_at, notes, created_by
purchase_items	id, purchase_id, variant_id, quantity, unit_cost, line_total
supplier_payments	id, supplier_id, payment_method_id, amount, paid_at, cash_session_id, reference, notes, created_by
supplier_payment_allocations	payment_id, purchase_id, amount
supplier_ledger (ledger)	id, supplier_id, entry_type (purchase, payment, credit_note, adjustment, opening_balance), amount (con signo), reference_type, reference_id, entry_date, notes, created_by

3.7 Importación Excel

Tabla	Campos clave
import_batches	id, kind (catalog, stock), mode, file_name, status (uploaded, validated, applied, failed), total_rows, ok_rows, warning_rows, error_rows, created_by, applied_at, summary
import_batch_rows	id, batch_id, row_number, raw, parsed, status (ok, warning, error), errors, action (create, update, skip), variant_id

3.8 Relaciones principales

```
categories, brands, suppliers --< products --< product_variants >-- colors, sizes
product_variants -- variant_costs (1:1)
product_variants -- stock_levels (1:1) <-- trigger -- stock_movements
product_variants --< sale_items >-- sales >-- cash_sessions >-- cash_registers
product_variants --< purchase_items >-- purchases >-- suppliers
product_variants --< inventory_count_items >-- inventory_counts
sales --< sale_payments >-- payment_methods --< cash_movements >-- cash_sessions
expenses / supplier_payments (si salen de caja) --> cash_movements
purchases / supplier_payments --> supplier_ledger
```

4. Cómo funciona el stock

Fuente de verdad: `stock_movements`.

1. Cada cambio de stock es un `INSERT` en `stock_movements` hecho por una función RPC. No existe otro camino.
2. Un trigger `AFTER INSERT` bloquea la fila de `stock_levels` (`FOR UPDATE`), calcula `stock_before` y `stock_after`, valida que no quede negativo (salvo `allow_negative_stock = true`) y actualiza `quantity`.
3. Un trigger `BEFORE UPDATE OR DELETE` en `stock_movements` lanza excepción.
4. `stock_levels` tiene `UPDATE` revocado para la API; solo el trigger (`SECURITY DEFINER`) lo toca.
5. **Integridad:** la vista `v_stock_audit` compara `stock_levels.quantity` con `SUM(stock_movements.quantity)` por variante; debe dar cero diferencias siempre (con test `pgTAP`).
6. **Concurrencia:** las funciones bloquean variantes en orden de `id` para evitar deadlocks. Si dos vendedores quieren la última prenda, uno recibe un mensaje claro y el stock no queda negativo.

Costo: al recibir una compra se recalcula `avg_cost` (promedio ponderado) y `last_cost`. Cada venta guarda el costo del momento en `sale_item_costs`.

Ajustes manuales: requieren motivo obligatorio y solo los hacen roles autorizados.

5. Cómo funciona el inventario físico

1. **Crear** un conteo (todo el local, una categoría o una selección). Se saca una foto: `expected_qty` por variante.
2. **Contar:** se carga `counted_qty` por variante (UI o Excel), con `counted_at` y quién contó. En el futuro el lector EAN cargará acá.
3. **Revisión:** diferencias (faltantes y sobrantes) valorizadas a costo; se pueden recontar ítems.
4. **Aplicar:** una RPC genera un movimiento `inventory_adjustment` por cada diferencia. El conteo pasa a `applied` y queda inmutable.

Como se sigue vendiendo durante el conteo, la diferencia no es `counted - expected` sino:

```
ajuste = counted_qty - stock_al_momento_de_contar
```

El stock a esa hora sale del `stock_after` del último movimiento anterior a `counted_at`. Así las ventas hechas durante el conteo no se cuentan como faltante.

6. Cómo funcionan las ventas

Se registran con una sola llamada: `register_sale(items, payments, customer, client_request_id)`. Dentro de una transacción:

1. Verifica permiso y que exista una **sesión de caja abierta**. Sin caja abierta no se vende.
2. Verifica idempotencia con `client_request_id` (un doble clic no duplica la venta).
3. Toma el precio vigente de cada variante y aplica descuentos (permitidos por rol/tope).
4. Valida que `SUM(payments) = total`. Se permite pago mixto (ej. efectivo + transferencia).
5. Crea `sales`, `sale_items` (con snapshot), `sale_item_costs` y `sale_payments`.
6. Genera un `stock_movements sale_out` por ítem.
7. Genera un `cash_movements` por cada pago, con `affects_physical_cash` según el método. Si el cliente paga con un billete mayor, solo el neto (monto menos vuelto) entra a caja.

Las ventas no se editan ni se borran: - `void_sale(sale_id, reason)`: anula, repone stock (`sale_void_in`) y registra los egresos de caja. Solo encargado o dueño. - `register_return(...)`: devolución o cambio parcial; se elige si vuelve a stock y por qué medio se reembolsa. Reembolso en efectivo sale de caja; por transferencia no toca el efectivo.

7. Cómo funciona la caja

Regla de oro: cada movimiento guarda su método de pago y solo los métodos con `is_cash = true` afectan el efectivo físico. Transferencia, débito, crédito y Mercado Pago nunca entran a la cuenta de efectivo. Se refuerza con un `CHECK/trigger`: `affects_physical_cash` debe coincidir con `payment_methods.is_cash`.

Ciclo de vida 1. **Apertura:** `open_cash_session(register, opening_amount)`; registra el dinero inicial como `opening_float`. Una sola sesión abierta por caja. 2. **Durante el turno:** ventas, ingresos manuales, gastos desde caja, retiros (con motivo obligatorio) y pagos a proveedores en efectivo. Todo queda como `cash_movements`. 3. **Cierre:** `close_cash_session(session, counted_cash, notes)`.

```
efectivo_esperado = dinero_inicial
                  + suma de ingresos en efectivo (ventas, ingresos manuales)
                  - suma de egresos en efectivo (gastos, retiros, pagos a proveedores,
devoluciones)

diferencia = efectivo_real_contado - efectivo_esperado
```

El cierre guarda además la foto por método de pago en `cash_session_totals`. Transferencias, débito, crédito y Mercado Pago aparecen **separados**, sin sumarse al efectivo. Opcionalmente se carga lo informado por la terminal para conciliar.

- Al cerrar, la sesión queda inmutable; la diferencia se guarda, no se "corrige".
- Reabrir una sesión cerrada solo lo puede hacer el dueño, con motivo, y queda en `audit_log`.
- **Gastos:** pagados en efectivo desde la caja generan un egreso; pagados por transferencia o desde otra cuenta quedan registrados pero **no tocan el efectivo**.

8. Cómo funcionan las compras

1. **Borrador:** proveedor, factura, fecha, vencimiento, ítems (variante, cantidad, costo unitario), descuentos e impuestos.

2. **Recepción** (`receive_purchase`): genera `purchase_in` por ítem, recalcula `avg_cost` y `last_cost`, e inserta en `supplier_ledger` una entrada `purchase` (aumenta la deuda).
3. **Pago**: pagada (pago por el total), parcial (pago menor, el resto pendiente) o pendiente (sin pago, a cuenta corriente hasta el vencimiento).
4. `payment_status` se actualiza solo según `paid_amount` frente a `total`.
5. **Anulación**: movimiento inverso (`purchase_return_out`) y nota de crédito en el ledger; no se borra.

Una orden de compra formal con recepción parcial se puede agregar más adelante.

9. Cuenta corriente de proveedores

Ledger con signo por proveedor. **Saldo = suma de amount** (positivo = lo que se le debe).

Evento	Entrada en el ledger
Recepción de compra	+ total
Pago al proveedor	- monto
Nota de crédito / devolución	- monto
Ajuste manual (con motivo)	+/- monto
Saldo inicial al migrar	+ monto

- `register_supplier_payment(supplier, method, amount, allocations?)` registra el pago y lo aplica a compras específicas; sin indicación aplica **FIFO** (la más antigua primero). Actualiza `paid_amount` y `payment_status`.
- Pago en efectivo desde la caja abierta: genera además un egreso `supplier_payment` en `cash_movements`. Por transferencia no toca el efectivo.
- **Vistas**: `v_supplier_balances`, estado de cuenta con saldo acumulado, y antigüedad de deuda (0-30, 31-60, 61+ días).

10. Importación y exportación de Excel

Exportación. El `.xlsx` trae una hoja `Stock` con una fila por variante: `variant_id` (oculta, para match exacto), `product_code`, `product_name`, `category`, `brand`, `color`, `size`, `sku`, `ean`, `cost` (solo roles autorizados), `price`, `stock_actual`, `stock_min`, `active`; más una hoja `Instrucciones` y una hoja `Listas` (colores, talles, categorías válidos). `ean` y `sku` se escriben como **texto** para que Excel no los pase a notación científica ni borre ceros a la izquierda.

Importación en 3 pasos, nunca directa 1. **Subir**: el navegador lee el archivo (SheetJS) y manda las filas a `import_batch_rows` (staging). 2. **Validar y previsualizar**: una RPC marca cada fila `ok`, `warning` o `error`. Controla EAN (formato y dígito verificador), EAN o SKU duplicados, talles y colores inexistentes, números negativos y filas repetidas. La UI muestra qué se crea, actualiza u omite y permite descargar los errores. 3. **Aplicar**: `apply_import(batch_id)` corre atómico. Con errores bloqueantes no se aplica nada (o solo las filas válidas si el usuario lo elige).

Orden de match de cada fila: `variant_id` -> **EAN** -> **SKU** -> (`product_code` + color + talle).

Dos modos - Catálogo: crea o actualiza productos y variantes (precio, costo, mínimos, EAN). - **Stock**: compara `stock_actual` del archivo con el del sistema y genera movimientos `import_adjustment` **solo por la diferencia**; nunca pisa el número directamente. La carga inicial usa `initial_load`.

Cada lote queda registrado con quién, cuándo y qué cambió.

11. Preparación para EAN

El EAN existe desde la primera migración, aunque no haya lector:

- `product_variants.ean` es `text` (nunca número), nullable, con índice único parcial (`UNIQUE WHERE ean IS NOT NULL`).
- `CHECK` de formato (solo dígitos, largo 8/12/13/14) y función SQL `is_valid_gtin(ean)` con dígito verificador. Se valida igual en `src/lib/ean.ts`.
- Una sola búsqueda sirve para todo: `find_variant_by_code(code)` busca por EAN y luego por SKU; la usan el POS, el inventario y el Excel.
- **El POS tendrá un único campo de búsqueda/escaneo.** Los lectores USB funcionan como teclado (tipean el código y envían Enter). Al sumar el lector no habrá que cambiar modelo, funciones ni Excel; solo se afina la UI (foco automático, sonido, error de código desconocido).
- El Excel ya incluye la columna `ean` y matchea por ella.
- **Futuro:** EAN internos (prefijo 2xx), etiquetas, y varios códigos por variante (`variant_barcodes`).

12. Roles y permisos

Permiso	Dueño	Encargado	Vendedor/Cajero	Depósito	Solo lectura
Vender, devolver	Sí	Sí	Sí	No	No
Abrir/cerrar caja propia	Sí	Sí	Sí	No	No
Ver todas las cajas	Sí	Sí	Solo la propia	No	Sí
Anular ventas	Sí	Sí	No	No	No
Descuentos sobre el tope	Sí	Sí	No	No	No
Gastos y retiros	Sí	Sí	Gastos chicos	No	No
Productos y precios	Sí	Sí	No	No	No
Ver costos y márgenes	Sí	Sí	No	Sí (costo)	Sí
Ajustes de stock / inventario	Sí	Sí	No	Sí	No
Importar Excel	Sí	Sí	No	Sí (stock)	No
Exportar Excel	Sí	Sí	No	Sí	Sí
Compras y proveedores	Sí	Sí	No	Recepción	Sí
Pagos a proveedores	Sí	Sí	No	No	No
Reportes	Sí	Sí	Solo ventas propias	No	Sí
Usuarios, configuración, reabrir caja	Sí	No	No	No	No

Cómo se aplica - `profiles.role` como enum y funciones helper (`auth_role()` , `has_role(...)`). -

RLS en todas las tablas. Las tablas transaccionales **no tienen políticas de**

INSERT/UPDATE/DELETE: solo se escribe por RPC. - Las RPC `SECURITY DEFINER` fijan `search_path` y **verifican el rol adentro**; nunca confían en el frontend. - Los guards de React son solo comodidad; la seguridad real está en la base. - La service key vive únicamente en la Edge Function. - Registro público desactivado: solo el dueño crea usuarios.

13. Orden recomendado de desarrollo

Etapa	Contenido	Por qué en este orden
0. Fundación	Repo, Vite + TS, lint, Supabase local, CI, docs	Base reproducible
1. Auth y roles	profiles , login, RLS base, audit_log , alta de usuarios	Todo depende de la seguridad
2. Catálogo	Colores, talles, categorías, productos, variantes, EAN, costos, historial de precios	Sin catálogo no hay nada
3. Stock	Ledger, stock_levels , triggers, ajustes manuales, v_stock_audit , tests	Núcleo de integridad
4. Excel	Exportar, importar catálogo y stock, carga inicial	Necesario para cargar el local real
5. Inventario físico	Conteos, diferencias, aplicar	Se apoya en stock
6. Caja	Métodos de pago, sesiones, movimientos, gastos, cierre	Las ventas la necesitan
7. Ventas (POS)	register_sale , pagos mixtos, anular, devolver	Integra stock y caja
8. Proveedores y compras	Compras, recepción, cuenta corriente, pagos	Integra stock, caja y ledger
9. Reportes	Ventas, margen, stock valorizado, caja, deudas, stock bajo y sin rotación	Con datos reales
10. Endurecimiento	E2E, backups, performance; luego EAN/lector y etiquetas	Cierre y futuro

Cada etapa cierra con migraciones, tests (pgTAP para reglas críticas) y una pantalla usable.

14. Decisiones pendientes (con propuesta por defecto)

1. **¿Un solo local o varios?** Propuesta: uno. Sumar sucursales implicaría agregar `location_id` a `stock_levels`, `cash_registers` y `sales`, sin rehacer el modelo.
2. **¿Cuántas cajas físicas?** Propuesta: el modelo soporta varias; se arranca con una.
3. **¿Facturación fiscal (ARCA/AFIP) o tickets?** Propuesta: fuera de alcance por ahora; solo comprobante interno.
4. **¿Clientes, cuenta corriente de clientes, señas o reservas?** Propuesta: `customers` mínimo y opcional; sin cuenta corriente de clientes.
5. **¿Recargos por cuotas o tarjeta de crédito?** Propuesta: se guarda `installments` pero sin cálculo automático de recargo.
6. **¿Stock negativo permitido?** Propuesta: no, configurable en `app_settings`.
7. **Cantidad de usuarios y roles.** Propuesta: los 5 roles de la tabla.